LAPORAN PRAKTIKUM

STRUKTUR DATA

“Abstract Data Type”

disusun Oleh:

Sofian Arba’i

2511533029

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI i
KATA PENGANTAR ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 1
BAB II PEMBAHASAN 3
2.1 Pengertian ADT 3
2.2 Implementasi ADT pada Java 4
2.1.1 Class Inti (Jam_2511533029) 4
2.1.2 Class Driver Sederhana (JamDriver_2511533029) 7
2.1.3 Class Driver Interaktif (JamDriver2_2511533029) 8
BAB III PENUTUP 11
3.1 Kesimpulan 11
3.2 Saran 11
DAFTAR PUSTAKA 12

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji syukur kehadirat Allah SWT karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan Laporan Praktikum Pekan 1 dengan judul "Abstract Data Type" ini dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum serta sebagai sarana untuk meningkatkan pemahaman mahasiswa mengenai ADT.

Dalam praktikum pekan ini, penulis mempelajari dasar-dasar Abstract Data Type, yang merupakan konsep penting dalam struktur data untuk mendefinisikan tipe data berdasarkan perilaku (operasi) yang dapat dilakukan terhadap data tersebut. Penyusunan laporan ini diharapkan dapat membantu penulis maupun pembaca dalam memahami materi secara lebih mendalam.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen pengampu mata kuliah Praktikum Strukdtur Data yang telah memberikan ilmu, arahan, serta bimbingannya.
2. Asisten praktikum yang telah membimbing dan memberikan penjelasan selama kegiatan praktikum berlangsung.

Saya meminta maaf jika banyak terdapat kesalahan pada penulisan laporan ini. Oleh karena itu, saya menerima kritik dan saran yang membangun demi perbaikan di masa mendatang.

Padang, 31 Maret 2026

Sofian Arba'i

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya pengolahan data yang efisien dan terstruktur. Dalam pemrograman, pengelolaan data yang baik sangat diperlukan agar suatu program dapat berjalan dengan optimal dan mudah dikembangkan. Salah satu konsep penting yang mendukung hal tersebut adalah Abstract Data Type (ADT).

ADT merupakan suatu konsep yang mendefinisikan tipe data berdasarkan operasi-operasi yang dapat dilakukan terhadap data tersebut, tanpa memperhatikan bagaimana implementasinya secara detail. Dengan menggunakan ADT, programmer dapat memisahkan antara konsep (abstraksi) dan implementasi, sehingga kode program menjadi lebih terstruktur, mudah dipahami, dan mudah untuk dikembangkan.

Oleh karena itu, pemahaman mengenai ADT sangat penting bagi mahasiswa, khususnya dalam bidang struktur data dan pemrograman. Melalui praktikum ini, diharapkan mahasiswa dapat memahami konsep dasar ADT serta mampu mengimplementasikannya dalam berbagai kasus pemrograman.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 1 tentang ADT adalah sebagai berikut:

1. Memahami konsep dasar Abstract Data Type (ADT).
2. Melatih kemampuan dalam merancang dan mengembangkan program berbasis ADT.
3. Memahami langkah penulisan kode program ADT

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 1 tentang ADT adalah sebagai berikut:

1. Mahasiswa dapat memahami konsep ADT secara teoritis maupun praktis.
2. Menjadi dasar dalam pengembangan aplikasi yang lebih besar dan kompleks.
3. Melatih pola pikir logis dan sistematis dalam menyelesaikan permasalahan pemrograman.

BAB II
PEMBAHASAN

2.1 Pengertian ADT

ADT adalah definisi TYPE dan sekumpulan PRIMITIF (operasi dasar) terhadap TYPE tersebut. Selain itu, dalam sebuah ADT yang lengkap, disertakan pula definisi invarian dari TYPE dan aksioma yang berlaku. ADT merupakan definisi statik. Definisi type dari sebuah ADT dapat mengandung sebuah definisi ADT lain.

Misalnya:

- ADT Waktu yang terdiri dari ADT JAM dan ADT DATE
- GARIS yang terdiri dari dua buah POINT

Primitif, dalam konteks prosedural, diterjemahkan menjadi fungsi atau prosedur. Primitif dikelompokkan menjadi :

- Konstruktor/Kreator, pembentuk nilai type. Semua objek (variabel) bertype tersebut harus melalui konstruktor. Biasanya namanya diawali Make.
- Selektor, untuk mengakses komponen tye (biasanya namanya diawali dengan Get)
- Prosedur, pengubah nilai komponen (biasanya namanya diawali Get)
- Validator komponen type, yang dipakai untuk meng_test apakah dapat membentuk type sesuai dengan batasan.
- Destruktor/Dealokator, yaitu untuk “menghancurkan” nilai objek (sekaligus memori penyimpanannya)
- Baca/Tulis, untuk interface dengan input/output device.
- Operator relational, terhadap type tersebut untuk mendefinisikan lebih besar, lebih kecil, sama dengan, dan sebagainya.
- Aritmatika, terhadap type tersebut karena biasanya aritmatika dalam bahasa pemrograman hanya terdefinisi untuk bilangan numerik.

2.2 Implementasi ADT pada Java

Pada praktikum pekan 1 mahasiswa diminta untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di papan tulis. Kode program tentang jam, berikut di bawah ini kode program yang dituliskan Kembali ke dalam java.

2.1.1 Class Inti (Jam_2511533029)

Class ini merupakan implementasi ADT (Abstract Data Type) Jam yang menjadi fondasi utama program. Class ini:

Berikut kode program dari class inti

```
package pekan1_2511533029;
public final class Jam_2511533029 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59
    // --- Validator ---
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <=
s && s <= 59);
    }
    // --- Konstruktor/Kreator ---
    public Jam_2511533029(int h, int m, int s) {
        this.hh = h;
        this.mm = m;
        this.ss = s;
    }
    // --- Selektor ---
    public int getHour() { return hh; }
    public int getMinute() { return mm; }
```

```
    public int getSecond() { return ss; }
    // --- Mutator ---
    public void setHour(int h) {
        if (0 <= h && h <= 23) this.hh = h;
    }
    public void setMinute(int m) {
        if (0 <= m && m <= 59) this.mm = m;
    }
    public void setSecond(int s) {
        if (0 <= s && s <= 59) this.ss = s;
    }
    // --- Konversi ---
    public int toSecond() {
        return hh * 3600 + mm * 60 + ss;
    }
    public static Jam_2511533029 fromSecond(int total) {
        if (total < 0) throw new IllegalArgumentException("detik
negatif");
        total %= 24 * 3600; // wrap 24 jam
        int h = total / 3600;
        total %= 3600;
        int m = total / 60;
        int s = total % 60;
        return new Jam_2511533029(h, m, s);
    }
    // --- Relasional ---
    public int compareTo(Jam_2511533029 other) {
        return Integer.compare(this.toSecond(), other.toSecond());
    }
    public boolean equals(Object o) {
        if (!(o instanceof Jam_2511533029 j)) return false;
```

```
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }
    public int hashCode() {
        return java.util.Objects.hash(hh, mm, ss);
    }
    // --- Aritmatika ---
    public Jam_2511533029 plus(Jam_2511533029 other) {
        return fromSecond(this.toSecond() + other.toSecond());
    }
    public Jam_2511533029 minus(Jam_2511533029 other) {
        return fromSecond(Math.floorMod(this.toSecond() - other.toSecond(), 24 * 3600));
    }
    public Jam_2511533029 nextSecond() {
        return fromSecond(this.toSecond() + 1);
    }
    public Jam_2511533029 nextNSecond(int n) {
        return fromSecond(this.toSecond() + Math.max(0, n));
    }
    public Jam_2511533029 prevSecond() {
        return fromSecond(Math.floorMod(this.toSecond() - 1, 24 * 3600));
    }
    //PrevNSeconds
    public Jam_2511533029 prevNSecond(int n) {
        return fromSecond(Math.floorMod(this.toSecond() - Math.max(0, n), 24 * 3600));
    }
    // --- Durasi (detik) ---
    public static int durasiDetik(Jam_2511533029 jaw, Jam_2511533029 jakh) {
```

```
        return jakh.toSecond() - jaw.toSecond();
    }
    // --- Override toString ---
    @Override
    public String toString() {
        return String.format("%02d:%02d:%02d", hh, mm, ss);
    }
}
```

- Validator: Metode statis untuk memastikan input waktu berada dalam rentang valid (jam 0-23, menit/detik 0-59).
- Konstruktor: Membentuk objek Jam baru dengan menginisialisasi atribut jam, menit, dan detik.
- Selektor & Mutator: Metode get untuk mengambil nilai dan set untuk mengubah nilai atribut dengan validasi internal.
- Konversi: Mengubah objek Jam ke total detik (toSecond) atau sebaliknya (fromSecond) untuk memudahkan perhitungan.
- Relasional: Membandingkan dua objek waktu (compareTo) atau mengecek kesamaan nilai (equals dan hashCode).
- Aritmatika: Operasi matematika waktu seperti penjumlahan (plus), pengurangan (minus), dan navigasi detik (nextSecond).
- toString: Mengubah objek menjadi format teks "hh:mm:ss" agar mudah ditampilkan ke layar.

2.1.2 Class Driver Sederhana (JamDriver_2511533029)

Berikut adalah kode program dari class driver sederhana

```
package pekan1_2511533029;
public class JamDriver_2511533029 {
    public static void main(String[] args) {
```

```
        Jam_2511533029 a = new Jam_2511533029(23, 59, 50);
        Jam_2511533029 b = new Jam_2511533029(0, 0, 15);
        System.out.println("a            = " + a);
        System.out.println("b            = " + b);
        System.out.println("a + b        = " + a.plus(b));
        System.out.println("next 20s     = " + a.nextNSecond(20));
        System.out.println("durasi(a,b)                    =      "      +
Jam_2511533029.durasiDetik(a, b));
        System.out.println("a.compareTo(b) = " + a.compareTo(b));
    }
}
```

Berikut penjelasan singkat untuk kode program di atas:

- Fungsi Utama: Menguji class inti secara cepat (*quick testing*) tanpa interaksi user.
- Input Hardcoded: Nilai waktu ditentukan langsung di dalam kode untuk memastikan konsistensi hasil testing.
- Demo Operasi: Menampilkan hasil operasi dasar seperti penjumlahan waktu, durasi, dan perbandingan ke layar.

2.1.3 Class Driver Interaktif (JamDriver2_2511533029)

Berikut adalah kode program dari class driver interaktif

```
package pekan1_2511533029;
import java.util.Scanner;
public class JamDriver2_2511533029 {
    public static void main(String[] args) {
            Scanner input =new Scanner (System.in);
            System.out.println("=== Program Driver Objek Jam ===");
            //input jam pertama
            System.out.println("\n--- Input Jam 1 ---");
            Jam_2511533029 j1= buatJamDariInput(input);
```

```
//input jam kedua
System.out.println("\n--- Input Jam 2 ---");
Jam_2511533029 j2= buatJamDariInput(input);
//menampilkan data
System.out.println("\n--- Hasil Operasi ---");
System.out.println("Jam 1 (String          : " + j1.toString());
System.out.println("Jam 2 (String          : " + j2.toString());
System.out.println("Jam 1 dalam detik     : " + j1.toSecond());
System.out.println("Jam 2 dalam detik     : " + j2.toSecond());
//operasi relasional
int perbandingan= j1.compareTo(j2);
if (perbandingan > 0) {
System.out.println(" Status                : Jam 1 lebih    lambat
(setelah) jam 2");
} else if (perbandingan <0) {
System.out.println("Status                    : Jam 1 lebih awal
(sebelum ) jam 2");
} else {
        System.out.println("Status                : jam 1 dan jam
2 sama persis");
}
//5operasi aritmatika
System.out.println("durasi  (j1  ke  j2)                    :  "  +
Jam_2511533029.durasiDetik(j1,j2) + "detik");
Jam_2511533029 jNext = j1.nextSecond();
System.out.println("jam 1 detik berikutnya      : " + jNext);
Jam_2511533029 jPrev = j1.prevSecond();
System.out.println("jam 1 detik sebelumnya      : "+ jPrev);
//6 penjumlahan
Jam_2511533029 jHasilPlus= j1.plus(j2);
System.out.println("hasil j1+j2        : " + jHasilPlus);
```

```
        input.close();
        System.out.println("\nprogram selesai");
    }
        private static Jam_2511533029  buatJamDariInput(Scanner
sc) {
            int h, m, s;
            while (true) {
            System.out.println("masukkan jam (0-23): ");
                h = sc.nextInt();
            System.out.println("masukkan menit (0-59): ");
                m = sc.nextInt();
            System.out.println("masukkan menit (0-59)  : ");
                s = sc.nextInt();
                //memanggil method
                if (Jam_2511533029.isValid(h, m, s)) {
                    return new Jam_2511533029 (h,m,s);
                    } else {
            System.out.println("[erorr] input tidak valid!
silahkan ulangi.\n");
                    }
            }
        }
}
```

Berikut penjelasan singkat untuk kode program di atas:

- Input User: Menerima masukan waktu dari pengguna secara dinamis menggunakan Scanner.
- Validasi Loop: Mengulang proses input melalui method helper hingga data yang dimasukkan valid.
- Demo Lengkap: Menampilkan seluruh fitur ADT (konversi, relasi, aritmatika) secara komprehensif berdasarkan input user.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan pelaksanaan praktikum Struktur Data pekan 1 mengenai Abstract Data Type (ADT), dapat disimpulkan bahwa konsep ADT berhasil diimplementasikan melalui pembuatan class Jam_2511533029 yang memisahkan antara definisi tipe data dan operasi-operasinya secara jelas. Praktikum ini membuktikan bahwa penggunaan prinsip enkapsulasi melalui atribut private dan method public mampu menjaga integritas data waktu serta memudahkan pengelolaan berbagai operasi seperti konversi detik, relasi perbandingan, dan aritmatika waktu. Selain itu, pembuatan class driver sederhana (JamDriver) dan interaktif (JamDriver2) menunjukkan pentingnya tahap pengujian untuk memverifikasi fungsionalitas class inti, sehingga tujuan praktikum untuk memahami konsep dasar ADT serta mampu merancang program berbasis ADT dalam bahasa Java telah tercapai dengan baik.

3.2 Saran

Untuk praktikum selanjutnya, diharapkan materi dapat dikembangkan dengan studi kasus ADT yang lebih bervariasi agar mahasiswa semakin terlatih dalam menerapkan konsep abstraksi data pada berbagai permasalahan. Bagi mahasiswa, disarankan untuk lebih aktif berdiskusi dengan asisten praktikum ketika menemui kendala, serta membiasakan diri mempelajari materi sebelum praktikum dimulai agar proses pembelajaran dapat berjalan lebih efektif dan maksimal.

DAFTAR PUSTAKA

[1] Yudi Herdiana, “Abstract Data Type (ADT),”. [Daring]. Tersedia pada: Abstract Data Type (ADT) - Struktur Data [Diakses: 31-Mar-2026].

[2] Gandi Rifansyah, “Abstract Data Type (ADT),”. [Daring]. Tersedia pada: SEMUA ILMU: Abstract Data Type (ADT) [Diakses: 31-Mar-2026].

LAPORAN TUGAS PRAKTIKUM

Struktur Data

Pekan 1

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Laporan Tugas

Tugas Praktikum Pekan 1

- Buatlah sebuah program ADT mobil yang memiliki atribut: nama, tahun, cc, harga, dan merk
- Contoh: Avanza, 2020, 1300, 150000000, Toyota
- Buatlah method ADT mobil tersebut: tambah mobil, hapus mobil, selektor dan mutator
- Buatlah 2 kelas: 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas lengkapi dengan NIM

Jawaban

Berikut kode program ADT mobil

A. Class Utama (Mobil_2511533029)

```java
package pekan1_2511533029;

public class Mobil_2511533029 {
    // --- Atribut ---
    private String nama;
    private int tahun;
    private int cc;
    private long harga;
    private String merk;

    // --- Validator ---
    public static boolean isValid(int tahun, int cc, long harga) {
        return (tahun >= 1900 && tahun <= 2026) &&
               (cc > 0 && cc <= 10000) &&
               (harga > 0);
    }

    // --- Konstruktor / Kreator ---
    public Mobil_2511533029(String nama, int tahun, int cc, long harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }

    // --- Selektor (Getter) ---
    public String getNama() { return nama; }
    public int getTahun() { return tahun; }
    public int getCc() { return cc; }
    public long getHarga() { return harga; }
    public String getMerk() { return merk; }

    // --- Mutator (Setter) ---
    public void setNama(String nama) { this.nama = nama; }
    public void setTahun(int tahun) {
```

Kode Program 1

```java
    // --- Selektor (Getter) ---
    public String getNama() { return nama; }
    public int getTahun() { return tahun; }
    public int getCc() { return cc; }
    public long getHarga() { return harga; }
    public String getMerk() { return merk; }

    // --- Mutator (Setter) ---
    public void setNama(String nama) { this.nama = nama; }
    public void setTahun(int tahun) {
        if (tahun >= 1900 && tahun <= 2026) this.tahun = tahun;
    }
    public void setCc(int cc) {
        if (cc > 0 && cc <= 10000) this.cc = cc;
    }
    public void setHarga(long harga) {
        if (harga > 0) this.harga = harga;
    }
    public void setMerk(String merk) { this.merk = merk; }

    // --- Relasional ---
    public int compareTo(Mobil_2511533029 other) {
        return Long.compare(this.harga, other.harga);
    }

    @Override
    public String toString() {
        return String.format("%s %s (%d) - %d cc - Rp %,d",
                merk, nama, tahun, cc, harga);
    }
}
```

Kode Program 2

Kode program di atas merupakan Abstract Data Type (ADT) Mobil yang menyimpan lima atribut privat (nama, tahun, cc, harga, merk) dengan prinsip enkapsulasi agar data hanya dapat diakses melalui method publik. Class ini dilengkapi method isValid() untuk memvalidasi input numerik (tahun 1900-2026, cc 1-10000, harga >0), konstruktor untuk membentuk objek baru, serta method selektor (get) dan mutator (set) untuk membaca dan mengubah nilai atribut dengan validasi internal guna menjaga konsistensi data. Selain itu, class ini menyediakan method compareTo() untuk membandingkan dua objek mobil berdasarkan harga, serta method toString() yang di-override untuk mengubah objek menjadi representasi teks terformat Merk Nama (Tahun) - CC cc - Rp Harga agar output ke layar lebih rapi dan mudah dibaca.

B. Class Driver (MobilDriver_2511533029)

```java
package pekan1_2511533029;
import java.util.Scanner;

public class MobilDriver_2511533029 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // --- ARRAY MANUAL ---
        Mobil_2511533029[] garage = new Mobil_2511533029[5];
        int count = 0;

        System.out.println("=== ADT MOBIL ===");

        // --- 1. TAMBAH MOBIL ---
        System.out.println("--- Tambah Mobil 1 ---");
        System.out.print("Nama: "); String n1 = sc.nextLine();
        System.out.print("Tahun: "); int t1 = sc.nextInt();
        System.out.print("CC: "); int c1 = sc.nextInt();
        System.out.print("Harga: "); long h1 = sc.nextLong();
        sc.nextLine();
        System.out.print("Merk: "); String m1 = sc.nextLine();

        if (Mobil_2511533029.isValid(t1, c1, h1)) {
            garage[count++] = new Mobil_2511533029(n1, t1, c1, h1, m1);
            System.out.println("✓ Mobil ditambahkan.\n");
        }

        // --- 2. TAMBAH MOBIL 2 ---
        System.out.println("--- Tambah Mobil 2 ---");
        System.out.print("Nama: "); String n2 = sc.nextLine();
        System.out.print("Tahun: "); int t2 = sc.nextInt();
        System.out.print("CC: "); int c2 = sc.nextInt();
        System.out.print("Harga: "); long h2 = sc.nextLong();
        sc.nextLine();
        System.out.print("Merk: "); String m2 = sc.nextLine();
```

Kode Program 3

```java
        if (Mobil_2511533029.isValid(t2, c2, h2) && count < 5) {
            garage[count++] = new Mobil_2511533029(n2, t2, c2, h2, m2);
            System.out.println("✓ Mobil ditambahkan.\n");
        }

        // --- 3. TAMPILKAN (Selektor + toString) ---
        System.out.println("--- Daftar Mobil ---");
        for (int i = 0; i < count; i++) {
            System.out.println((i+1) + ". " + garage[i]);
        }

        // --- 4. RELASIONAL ---
        if (count >= 2) {
            System.out.println("\n--- Bandingkan Harga ---");
            int cmp = garage[0].compareTo(garage[1]);
            if (cmp < 0) System.out.println(garage[0].getNama() + " lebih murah");
            else if (cmp > 0) System.out.println(garage[0].getNama() + " lebih mahal");
            else System.out.println("Harga sama");
        }

        // --- 5. MUTATOR  ---
        if (count > 0) {
            System.out.println("\n--- Edit Harga (Mutator) ---");
            System.out.println("Sebelum: " + garage[0]);
            garage[0].setHarga(garage[0].getHarga() + 5000000);
            System.out.println("Sesudah: " + garage[0]);
        }

        // --- 6. HAPUS MOBIL ---
        System.out.println("\n--- Hapus Mobil 1 ---");
        if (count > 0) {
            for (int i = 0; i < count - 1; i++) {
                garage[i] = garage[i + 1];
            }
            garage[count - 1] = null;
```

Kode Program 4

```java
            garage[count - 1] = null;
            count--;
            System.out.println("✓ Mobil dihapus.\n");
        }

        // --- 7. TAMPILKAN SISA ---
        System.out.println("--- Sisa Mobil ---");
        for (int i = 0; i < count; i++) {
            System.out.println((i+1) + ". " + garage[i]);
        }
        if (count == 0) System.out.println("Garage kosong.");

        sc.close();
        System.out.println("\nProgram Selesai.");
    }
}
```

Kode Program 5

Kode program di atas berfungsi sebagai program pengujian interaktif untuk class inti Mobil yang mengelola seluruh alur eksekusi melalui method main. Program ini menggunakan array manual Mobil[] dengan kapasitas tetap dan variabel count untuk menyimpan serta melacak jumlah objek mobil, mendemonstrasikan konsep dasar manipulasi array tanpa bergantung pada Collections Framework. Melalui Scanner, program menerima input dinamis dari pengguna untuk membentuk objek mobil baru yang divalidasi sebelum disimpan, kemudian menampilkan daftar mobil menggunakan method toString() yang dipanggil otomatis. Program juga menguji operasi relasional dengan compareTo() untuk membandingkan harga, mutator setHarga() untuk mengubah nilai atribut, serta logika penghapusan manual dengan menggeser elemen array ke kiri dan mengurangi counter. Seluruh alur diakhiri dengan penutupan resource Scanner untuk mencegah memory leak, menunjukkan implementasi lengkap primitif ADT (kreator, selektor, mutator, validator, relasional, dan dealokator) dalam skenario nyata yang sederhana dan terstruktur.

C. Output Kode Program

Berikut output dari kode program ADT Mobil

```
=== ADT MOBIL ===
--- Tambah Mobil 1 ---
Nama: FORD RANGER WILDTRAK
Tahun: 2026
CC: 1996
Harga: 839000000
Merk: FORD
✓ Mobil ditambahkan.

--- Tambah Mobil 2 ---
Nama: GR SUPRA
Tahun: 2025
CC: 2998
Harga: 986000000
Merk: TOYOTA
✓ Mobil ditambahkan.
```

Gambar 1

```
--- Daftar Mobil ---
1. FORD FORD RANGER WILDTRAK (2026) - 1996 cc - Rp 839,000,000
2. TOYOTA GR SUPRA (2025) - 2998 cc - Rp 986,000,000

--- Bandingkan Harga ---
FORD RANGER WILDTRAK lebih murah

--- Edit Harga (Mutator) ---
Sebelum: FORD FORD RANGER WILDTRAK (2026) - 1996 cc - Rp 839,000,000
Sesudah: FORD FORD RANGER WILDTRAK (2026) - 1996 cc - Rp 844,000,000

--- Hapus Mobil 1 ---
✓ Mobil dihapus.

--- Sisa Mobil ---
1. TOYOTA GR SUPRA (2025) - 2998 cc - Rp 986,000,000

Program Selesai.
```

Gambar 2